E. Sink

time limit per test: 3 seconds
memory limit per test: 256 megabytes

You are given a grid containing n rows and m columns. Every cell (i, j), located in the i-th row and j-th column, has a positive integer value associated with it $a_{i,j}$. Two cells are adjacent if and only if they share a common side in the grid.

You are allowed to construct *holes* at cells of your choice. A cell (x, y) is a *sink* if and only if it has a hole, or is adjacent to a sink (i, j) with $a_{x,y} \geq a_{i,j}$.

The *beauty* of a grid is defined as the minimum number of holes you need to construct so that every cell becomes a sink.

You have to determine the beauty of this grid.

You are also given q queries.

In a query, you are given three positive integers r, c, and x. The current value of cell (r, c) is decreased by x. After each query, determine the beauty of the grid considering no holes have been constructed yet.

It is guaranteed that after every query the value of each cell will remain positive.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \leq n, m \leq 2 \cdot 10^5$, $1 \leq n \cdot m \leq 2 \cdot 10^5$) — the number of rows and columns, respectively.

The following n lines contain m integers each; the j-th element in the i-th line $a_{i,j}$ is the number written in the j-th cell of the i-th row ($1 \leq a_{i,j} \leq 10^9$).

The next line contains a single integer q ($0 \leq q \leq 2 \cdot 10^5$) — the number of queries.

The following q lines contain 3 integers each — r, c, and x ($1 \leq r \leq n, 1 \leq c \leq m, 1 \leq x < 10^9$).

It is guaranteed that the sum of $n \cdot m$ and the sum of q over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed that after every query the value of each cell will remain positive.

Output

For each test case, print $q + 1$ lines.

On the first line print the beauty of the initial grid.

Also, after each query, print the beauty of the current grid.

Example

input

```
3
1 4
1 2 3 5
2
1 4 1
```


Codeforces Round 1067 (Div. 2)
Contest is running
01:13:52
Contestant

→ Submit?

Language: GNU G++17 7.3.0

Choose file: Choose File No file chosen

Be careful: there is 50 points penalty for submission which fails the pretests or resubmission (except failure on the first test, denial of judgement or similar verdicts). "Passed pretests" submission verdict doesn't guarantee that the solution is absolutely correct and it will pass system tests.

Submit

→ Score table

	Score
Problem A	410
Problem B	1025
Problem C	1230
Problem D	1640
Problem E	2255
Problem F1	2050
Problem F2	820
Successful hack	100
Unsuccessful hack	-50
Unsuccessful submission	-50
Resubmission	-50

* If you solve problem on 00:45 from the first attempt

```
1 3 2
3 3
5 1 6
2 9 3
7 4 8
3
2 2 1
2 2 7
3 3 7
3 4
10 10 10 10
10 10 10 10
10 10 11 10
5
3 3 5
2 2 5
2 4 5
2 3 5
1 1 9
```

output Copy

```
1
1
2
4
4
1
2
1
1
2
3
1
2
```

Note

For the first test case:

- Initially, we can create a hole at cell $(1, 1)$.
- After the first query, we can create a hole at cell $(1, 1)$.
- After the second query, we can create holes at cells $(1, 1)$ and $(1, 3)$.

For the second test case:

- Initially, we can create holes at cells $(1, 2)$, $(2, 1)$, $(2, 3)$, and $(3, 2)$.
- After the first query, we can create holes at cells $(1, 2)$, $(2, 1)$, $(2, 3)$, and $(3, 2)$.
- After the second query, we can create a hole at cell $(2, 2)$.
- After the third query, we can create holes at cells $(2, 2)$ and $(3, 3)$.